

Обработка исключений

Исключение — это аномальное поведение во время выполнения, которое программа может обнаружить, например: деление на 0, выход за границы массива, истощение свободной памяти. Такие исключения нарушают нормальный ход работы программы, и на них нужно немедленно отреагировать.

Все исключения в Платформе являются наследниками класса `sbis::Exception`, который позволяет сохранять код ошибки и сообщения об ошибке. Платформенные исключения бывают трех типов:

- **предупреждения**,
- **ошибки**,
- **критические ошибки**.

Если возникшая ошибочная ситуация связана с какими-либо неправильными с точки зрения системы действиями пользователя (например, пользователь ввел неверный пароль при аутентификации), либо неизбежна ввиду самой природы исполняемой операции на некоторых ветках исполнения алгоритма (например, пользователь сделал изменения на некоторой карточке, которая во время редактирования была удалена другим пользователем в параллельной сессии), то должно генерироваться исключение с типом **предупреждение** и понятным сообщением для пользователя, в котором будет указано что пошло не так и что ему теперь делать.

Все ошибочные ситуации, не предусмотренные на этапе разработки и проектирования, в ходе работы приложения считаются **ошибками**. Сюда относятся разного рода низкоуровневые сбои (деление на 0, ошибки разбора форматных файлов, криво сформированные запросы и т.п.), а также ошибки в использовании API сервисов (несовместимые комбинации параметров вызова методов БЛ, неверная последовательность вызовов, отсутствие необходимых авторизационных данных и т.п.).

Выброс исключений типа **ошибка** является признаком наличия багов в приложении, поэтому политика компании состоит в том, что ошибок быть не должно. Соответственно, при появлении в логах признаков исключения данного типа, — разработчикам, ответственным за сгенерировавший исключение метод, выписывается задача на выяснение источника проблемы.

Также существует выделенный класс сбоев — **критические ошибки**. К данному классу относятся ошибки отказа инфраструктуры приложения, в результате которых оно не может осуществлять свою деятельность. Информацию по критическим ошибкам можно посмотреть в документе [API Критические ошибки](#).

Например, невозможность установить соединение с базой данных или другим задействованным хранилищем, недоступность брокера обмена сообщениями и т.п. В системе реализован выделенный слой для оперативного мониторинга ошибок подобного класса на уровне службы эксплуатации, т.к. их появление, как правило, ведет к массовому отказу в обслуживании клиентов. Из прикладного кода генерировать ошибки подобного класса не следует, кроме случаев разработки под мобильные платформы, где исключения данного класса транслируются в [Crashlytics](#).

В C++ тип ошибки задается методом `SetType` класса `sbis::Exception`, в Python есть классы `sbis.Error` и `sbis.Warning`.

Сообщения об ошибках

Платформенные исключения хранят три сообщения об ошибке:

- внутреннее — предназначено для вывода в лог и т.п.,
- пользовательское — предназначено для отображения пользователю,
- подсказка — предназначено для отображения подсказки по ошибке.

Для получения сообщения об ошибке используются методы:

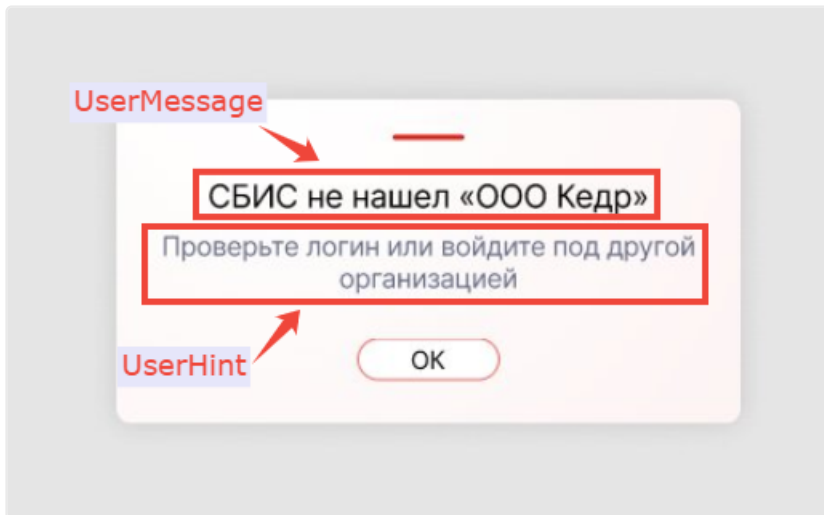
```
1 // C++
2 class Exception
3     ...
4     const String& ErrorMessage() const noexcept;
5     const String& ErrorUserMessage() const noexcept;
6     const String& ErrorUserHint() const noexcept;

1 # Python
2 class Error( Exception ):
3     ...
4     def ErrorMessage( self ):
```

```

5     def ErrorMessage( self ):
6     def ErrorUserHint( self ):

```



Обработка ошибок на UI описана в статье [Обработка ошибок в Wasaby](#).

Коды ошибок

Код ошибки предназначен для детализации ошибки внутри функциональной области. Код ошибки — 32-битное знаковое целое число.

Диапазон кодов разбит на следующие блоки:

1. Внутренние ошибки платформы.

Коды $x \leq \text{SBIS_PLATFORM_SYSTEM_ERROR_MAX}$

Сообщения в исключениях данного типа не предназначены для отображения пользователю. Это "низкоуровневые" ошибки платформы и прикладных сервисов.

2. Ошибки платформы.

Коды $\text{SBIS_PLATFORM_SYSTEM_ERROR_MAX} < x \leq \text{SBIS_PLATFORM_USER_ERROR_MAX}$

Ошибки этого типа должны содержать пользовательское сообщение и генерируются исключительно платформой.

3. Ошибки прикладных программ.

Коды $x > \text{SBIS_PLATFORM_USER_ERROR_MAX}$

Ошибки этого типа должны содержать пользовательское сообщение и генерируются исключительно прикладными сервисами.

Все константы импортируются из модуля **sbis**.

Для определения принадлежности используются методы:

```

1 // C++
2 bool IsPlatformSystemException() const noexcept;
3 bool IsPlatformUserException() const noexcept;
4 bool IsApplicationException() const noexcept;

1 # Python
2 class Error( Exception ):
3     ...
4     def IsPlatformSystemException( self ):
5     def IsPlatformUserException( self ):
6     def IsApplicationException( self ):

```

Стандартные коды ошибки, используемые, если в функциональной области не определены собственные коды ошибок:

```

1 // C++
2 constexpr SbisErrorCode EMPTY_SBIS_ERROR_CODE;
3 constexpr SbisErrorCode SBIS_GENERIC_PLATFORM_SYSTEM_ERROR;

```

```
4 constexpr SbisErrorCode SBIS_GENERIC_PLATFORM_USER_ERROR;
5 constexpr SbisErrorCode SBIS_GENERIC_APPLICATION_ERROR;
```

Примеры платформенных кодов ошибок

Запись при вызове метода "Прочитать" отсутствует.

```
NO_RETURN_RECORD = 32002
```

Генерация и перехват исключений

Генерация исключения без пользовательского сообщения:

```
1 // C++
2 String log_message, user_message;
3 throw Exception( SBIS_GENERIC_PLATFORM_SYSTEM_ERROR, log_message );

1 # Python
2 raise sbis.Error( log_message,
  error_code=SBIS_GENERIC_PLATFORM_SYSTEM_ERROR )
```

Генерация исключения с пользовательским сообщением:

```
1 // C++
2 String log_message, user_message;
3 throw Exception( log_message, user_message,
  SBIS_GENERIC_APPLICATION_ERROR );

1 # Python
2 raise sbis.Error( log_message, user_message,
  SBIS_GENERIC_APPLICATION_ERROR )
```

Сообщения об ошибках, выводимые пользователю, должны быть локализованы, о локализации можно прочитать [здесь](#).

Также для генерации исключений в Платформе реализованы вспомогательные функции:

- **Warning** — генерирует предупреждение и записывает в лог сообщение detail_msg с типом **warning**.

```
1 // C++
2 template< class E >
3 void Warning( const String& detail_msg, SbisErrorCode code =
  EMPTY_SBIS_ERROR_CODE );
4 template<class E>
5 void Warning( const String& detail_msg, const String& user_msg,
  SbisErrorCode code = EMPTY_SBIS_ERROR_CODE )
6 template< class E > // перегрузка со всплывающей подсказкой
7 [[noreturn]] void Warning( const String& detail_msg, const String&
  user_msg, const String& user_hint, SbisErrorCode code =
  EMPTY_SBIS_ERROR_CODE )
8 {
9     static_assert( std::is_base_of< Exception, E >::value, "Template
  parameter of this function must be sbis::Exception!" );
10    WarningMsg( detail_msg );
11    E ex( code, detail_msg, user_msg, user_hint );
12    ex.SetType( Exception::etWarning );
13    throw( ex );
14 }
```

- **Error** — генерирует ошибку и записывает в лог сообщение detail_msg с типом **error**.

```

1 // C++
2 template< class E >
3 void Error( const String& detail_msg, const String& user_msg,
  SbisErrorCode code = EMPTY_SBIS_ERROR_CODE );
4 template< class E >
5 void Error( const String& detail_msg, SbisErrorCode code =
  EMPTY_SBIS_ERROR_CODE );
6 template<class E> // перегрузка со всплывающей подсказкой
7 FUNC_ATTRIBUTE_NORETURN
8 void Error( const String& detail_msg, const String& user_msg, const
  String& user_hint, SbisErrorCode code = EMPTY_SBIS_ERROR_CODE )
9 {
10     static_assert( std::is_base_of< Exception, E >::value, "Exeption
  should be used as a template parameter of this function!" );
11     ErrorMessage( detail_msg );
12     throw Exception( code, detail_msg, user_msg, user_hint );
13 }

```

- **UserError** — генерирует предупреждение и записывает в лог сообщение user_msg с типом **warning**. В генерируемое исключение передается локализованное сообщение user_msg.

```

1 // C++
2 template< class E >
3 void UserError( SbisErrorCode code, const wchar_t* user_msg ;
4 template< typename ExceptionType >
5 void UserError( const wchar_t* user_msg )

```

Сообщение, возвращаемое методом ErrorUserMessage не имеет никакой ценности для пользователя, если оно принадлежит к внутренним ошибкам платформы, т.е. если метод IsPlatformSystemException возвращает true. Соответственно, если вы пишете код для обработки исключений и поймали исключение данного класса, то необходимо выдать "наружу" уже свое, достаточно обобщенное сообщение, без отображения данных из пойманного объекта.

Исключения и код возврата сервера БЛ

При выбросе необработанного исключения из метода БЛ его исполнение досрочно завершается. Обработкой выброшенного исключения занимается код Платформы. При этом для каждого такого ответа назначается HTTP-код возврата. Политика здесь такая:

- Если пойманное исключение имеет тип **предупреждение** и содержит пользовательское сообщение об ошибке, тогда сервер возвращает код 200 OK, иначе возвращается код 500 Internal Server Error.
- Исключения составляют специализированные типы исключений GenericHttpWarning и GenericHttpError.

Для них код возврата определяется соответствующим атрибутом экземпляра исключения.

Кроме того, в Платформе имеется ряд других типов исключений, имеющих выделенный код возврата.

Использовать их в прикладном коде не следует. Также сериализуемые исключения могут переопределять код HTTP.

Ошибки в межсервисных вызовах

Для того чтобы не терять тип исключения при возникновении ошибки в межсервисных вызовах, в платформе реализованы сериализуемые исключения. Сериализуемые исключения наследуются от класса sbis::rpc::BaseSerializableException. Они позволяют не только получить исключение того же типа, что было выброшено в удаленном вызове, но и дополнительно передать произвольный набор данных.

Сериализуемые исключения идентифицируются при помощи UUID, который должен быть зарегистрирован и на сервере, и на клиенте. Если исключение не зарегистрировано на клиенте, то будет выброшено исключение AbstractError.

Для корректной работы реализация сериализуемого исключения должна быть помещена в отдельный модуль, который будет добавлен в зависимости всех модулей, использующих данное исключение (как на сервере, так и на клиентах).

Создание сериализуемых исключений

Вручную

Для создания сериализуемого исключения нужно создать класс наследующийся от `sbis::rpc::BaseSerializableException` и зарегистрировать его при помощи макроса `REGISTER_SERIALIZABLE_EXCEPTION( ClassName )`.

Пример:

```
1 // C++
2 class TestExceptionWithData: public BaseSerializableException
3 {
4     typedef BaseSerializableException MyParent;
5 public:
6     TestExceptionWithData( const char* msg ) :
7         MyParent( msg ),
8         mId( boost::uuids::string_generator()( "{xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxxxxxx}" ) )
9     {
10    }
11    ...
12 private:
13     const boost::uuids::uuid mId;
14 };
15
16 REGISTER_SERIALIZABLE_EXCEPTION( TestExceptionWithData );

1 # Python
2 class TestExceptionWithData( SerializableException ):
3     StaticId = 'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx'
4     def __init__( self, details=detail_message, user_msg = "",
5 error_code = -1, type = ExceptionType.ERROR, passed_exception=None ):
6         super().__init__( details, user_msg, error_code, self.StaticId,
7 type, passed_exception )
8
9 ExceptionList().register( TestExceptionWithData )
```

Макросом

Если передача данных в сериализуемом исключении не требуется, то создание исключения можно упростить, используя макросы `DECLARE_SERIALIZABLE_EXCEPTION( ClassName, ClassId )` или `DECLARE_DERIVED_SERIALIZABLE_EXCEPTION( ClassName, BaseClassName, ClassId )`, если исключение наследуется не от `sbis::rpc::BaseSerializableException`.

Пример:

```
1 // C++
2 DECLARE_SERIALIZABLE_EXCEPTION( SimpleTestException, "{xxxxxxxx-xxxx-
xxxx-xxxx-xxxxxxxxxxxxxxxx}" );
```

В Python для упрощения создания сериализуемых исключений используется функция `CreateErrorClass`, принимающая в качестве параметров UUID и сообщение об ошибке по умолчанию.

```
1 # Python
2 TestException = CreateErrorClass( 'xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxxxx', 'Internal server error' )
3 ExceptionList().register( TestException )
```

Передача данных в сериализуемом исключении

В дополнение к сохранению типа, сериализуемые исключения позволяют передать внутри исключения произвольные данные. Для этого нужно перегрузить методы:

```
1 // C++
2 virtual void SerializeData( IObjectWriter& data_writer ) const = 0;
3 virtual void DeserializeData( IObjectReader& data_reader ) = 0;
```

Пример для передачи Int64:

```
1 // C++
2 class TestExceptionWithData: public BaseSerializableException
3 {
4     ...
5     virtual void SerializeData( IObjectWriter& data_writer ) const
6     {
7         WriteValue( data_writer, mValue );
8     }
9
10    virtual void DeserializeData( IObjectReader& data_reader )
11    {
12        ReadValue( data_reader, mValue );
13    }
14    ...
15 private:
16     Int64 mValue;
17 };
```

Пример

Сериализуемое исключение для ошибки парсинга JSON, в котором сохраняются имя файла, позиция в файле и текст ошибки.

```
1 // C++
2 namespace
3 {
4     const boost::uuids::uuid JSON_PARSE_ERROR_UUID = UUIDFromString( "
5     {8fb6ba83-dfdf-4803-b16d-21fe18a0c5e0}"_sv );
6 }
7 class JsonParseError : public rpc::BaseSerializableException
8 {
9 public:
10     JsonParseError( const String& detail_msg )
11         : BaseSerializableException( detail_msg )
12     {
13     }
14
15     JsonParseError( const String& detail_msg, const String& user_msg )
16         : BaseSerializableException( detail_msg, user_msg )
17     {
18     }
19
20     JsonParseError( SbisErrorCode error_code, const String& detail_msg,
21     const String& user_msg )
22         : BaseSerializableException( error_code, detail_msg, user_msg )
23     {
24     }
```

```
24
25     const boost::uuids::uuid& Id() const override
26     {
27         return JSON_PARSE_ERROR_UUID;
28     }
29
30     const String& File() const noexcept
31     {
32         return mFile;
33     }
34
35     void SetFile( String file )
36     {
37         mFile = std::move( file );
38     }
39
40     const String& Error() const noexcept
41     {
42         return mError;
43     }
44
45     void SetError( String error )
46     {
47         mError = std::move( error );
48     }
49
50     size_t Position() const noexcept
51     {
52         return mPos;
53     }
54
55     void SetPosition( size_t pos ) noexcept
56     {
57         mPos = pos;
58     }
59
60     void SerializeData( IObjectWriter& data_writer ) const override
61     {
62         data_writer.BeginWriteObject();
63         data_writer.BeginWriteObjectElem( L"file" );
64         data_writer.WriteElem( mFile );
65         data_writer.EndWriteObjectElem( L"file" );
66         data_writer.BeginWriteObjectElem( L"position" );
67         data_writer.WriteElem( mPos );
68         data_writer.EndWriteObjectElem( L"position" );
69         data_writer.BeginWriteObjectElem( L"error" );
70         data_writer.WriteElem( mError );
71         data_writer.EndWriteObjectElem( L"error" );
72         data_writer.EndWriteObject();
73     }
74
75     void DeserializeData( IObjectReader& data_reader ) override
76     {
77         if( data_reader.HasMember( L"file" ) )
78             data_reader.ReadString( mFile );
```

```

79         if( data_reader.HasMember( L"error" ) )
80             data_reader.ReadString( mError );
81         if( data_reader.HasMember( L"position" ) )
82             data_reader.Read( mPos );
83     }
84
85 private:
86     boost::uuids::uuid mId;
87     String mFile;
88     String mError;
89     size_t mPos;
90 };
91
92 REGISTER_SERIALIZABLE_EXCEPTION( JsonParseError );

```

Код возврата HTTP для сериализуемых исключений

По умолчанию для сериализуемых исключений используется код 500 Internal Server Error. При необходимости можно создать свой тип сериализуемого исключения, которое будет иметь свой собственный код HTTP.

Для этого нужно объявить и зарегистрировать свой класс, наследующий `grpc::BaseSerializableException` (или `SerializableException` для Python) и переопределить метод `HttpCode` (или `http_code` для Python), который возвращает `std::pair< int, StringView >` (или `tuple` для Python).

Пример

```

1  # Python
2  class TestWarning299( SerializableException ):
3      StaticId = 'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx'
4      def __init__(self, details='Warning', user_msg="", error_code=-1,
5                  type=ExceptionType.WARNING, passed_exception=None):
6          super().__init__(details, user_msg, error_code, self.StaticId,
7                          type, passed_exception)
8          def http_code(self):
9              return (299, '299 Warning')
10
11  ExceptionList().register( TestWarning299 )

```

Формирование дружественных ошибок и предупреждений

Данный тип ошибок/предупреждений используется в БД, когда запрос или триггер не отработал и нужно отобразить пользователю локализованный текст ошибки или предупреждения с рекомендацией по устранению (подсказкой), т.е. отдельно локализовать формат сообщения и аргументы формата.

Простейший способ вывода таких ошибок и предупреждений - использование команды `RAISE EXCEPTION` с префиксом в тексте сообщения:

```

1  RAISE EXCEPTION 'sbis_err::текст ошибки';
2  RAISE EXCEPTION 'sbis_wrn::текст предупреждения';

```

Однако при использовании этих команд **текст не переводится**.

Для перевода пользовательских ошибок и предупреждений, генерируемых в PL/pgSQL используются хранимые процедуры `sbis_raise_error` или `sbis_raise_warning` с переменным числом строковых аргументов:

```

1  sbis_raise_error('текст ошибки: {} {rk#{0}}', '12', 'яблоко(-а, -ов)');
2  sbis_raise_warning('текст предупреждения с параметрами: {1} {0}

```



```
{3:rk@контекст}', 'не переводится', 'тоже нет', 'этот переводится в контексте');
```

Эти процедуры, как и команда **raise exception**, вызывают исключение в PL/pgSQL, но со специально сформатированной строкой, в которой сериализованы форматная строка и все параметры.

Форматная строка и параметры пересылаются из БД в сервис БЛ вместе с информацией об исключении, и только там переводятся и подставляются для формирования конечного сообщения. Это необходимо для перевода форматной строки **до подстановки параметров**.

Строки для перевода должны содержаться в [словаре](#). Форматные строки функций **sbis_raise_error** и **sbis_raise_warning** собираются из файлов *.sql, *.trg и *.orgх утилитой [Jinnee](#). Дополнительные параметры, которые переводятся, в словарь автоматически не попадают, поэтому их нужно добавлять вручную.

Также, в дополнение к основному тексту ошибки, можно передать локализуемую подсказку. Для этого следует использовать перегрузки процедур **sbis_raise_error** и **sbis_raise_warning**, принимающие два аргумента типа массив строк:

```
1 sbis_raise_error(message => array ['Форматная строка "{}" сообщения',  
  'аргумент подстановки в сообщение'], hint => array ['Форматная строка  
  {} подсказки #{}', 'аргументы подстановки', 'в подсказку']);  
2  
3 sbis_raise_warning(message => array ['Транспортное средство с СТС "{}"  
  уже существует', reg_cert_number], hint => array ['Зарегистрировано {}  
  по документу #{}', reg_date, reg_doc_num]);
```

Такая подсказка отображает рекомендуемые действия по устранению ошибки.

Для выброса сериализуемых исключений из БД можно воспользоваться перегрузками, принимающими UUID, JSON, а также один или два текстовых параметра:

```
1 select sbis_raise_error('$UUID_сериализуемого_исключения'::uuid,  
  '$данные_сериализуемого_исключения'::jsonb, 'Сообщение для  
  пользователя'::text, 'Совет, что делать для пользователя'::text);  
2  
3 select sbis_raise_warning('$UUID_сериализуемого_исключения'::uuid,  
  '$данные_сериализуемого_исключения'::jsonb, 'Сообщение для  
  пользователя'::text, 'Совет, что делать для пользователя'::text);
```

Форматирование сообщения

Из строковых аргументов, принимаемых процедурами **sbis_raise_error** / **sbis_raise_warning** формируется сообщение об ошибке/предупреждении.

Первый аргумент – это формат сообщений, он обязательный. Если он начинается с двоеточия, то сам формат не переводится, двоеточие при форматировании убирается.

Последующие аргументы представляют собой подстановочные параметры для формирования сообщения. Подстановка параметров производится с помощью метода [format](#) в Python. Разрешена только спецификация формата [rk](#), она означает необходимость перевода этого параметра.

Дополнительно можно указать контекст @context и числительное, в соответствии с которым нужно выбрать форму слова, через #число (например, #123). Внутри спецификации разрешена однократная вложенность подстановок для указания контекста или числительного через параметры, например: {0:rk@{1}#{2}}.

Когда передаются два массива строк, первый формирует основной текст ошибки, а второй – подсказку. В массивах, аналогично варианту с переменным числом строковых аргументов, первый элемент – это форматная строка, последующие элементы – подстановочные параметры.

Примеры использования

```
1 sbis_raise_error('Партнера с кодом "{}" не существует', partnercod);  
2 sbis_raise_error('Неизвестный тип сообщений: {}', msgtyp);  
3 sbis_raise_warning('Ингредиент ({})) не может входить в свой собственный  
  состав!', _comp_nom_id);
```

Детали реализации сериализуемых исключений

Для понимания реализации сериализуемых исключений рассмотрим несколько сценариев передачи такого исключения.

Передача сериализуемого исключения от сервера до клиента на Python

Сериализуемое исключение, брошенное в коде метода БЛ на сервере, перехватывается в `rpc::ExecuteRequest`, сериализуется и передается на сторону клиента в ответе JSON-RPC или SOAP. UUID исключения кодируется в теле ответа в виде атрибута `classid`.

Далее на клиенте в `JsonResponseReader::DoProtocolParse` или `SoapResponseReader::DoProtocolParse`, при распознавании ошибочного ответа, вызывается `AbstractResponseReader::GenerateException`.

`AbstractResponseReader::GenerateException` вызывает `rpc::ExceptionList::Instance().GenerateException()`, где важную роль играет UUID исключения. Если исключение найдено по UUID среди зарегистрированных исключений, то вызывается соответствующий ему генератор исключений, иначе создается стандартное исключение `AbstractError`.

Для исключений, зарегистрированных из C++ с помощью макроса `REGISTER_SERIALIZABLE_EXCEPTION` или `DECLARE_DERIVED_SERIALIZABLE_EXCEPTION`, вызывается соответствующий конкретному классу исключения шаблонный генератор `rpc::SerializableExceptionGenerator`, а для исключений, зарегистрированных из Python вызывается общий генератор, который бросает исключение `py::PythonSerializableException`. В классе `PythonSerializableException` запоминается UUID исключения.

Для передачи исключений в Python регистрируются трансляторы исключений с помощью шаблонного метода `py::ExceptionTranslator::RegisterTranslator`. Все сериализуемые исключения транслируются единым транслятором `py::BaseSerializableException_Translate`, который вызывает метод Python'a `sbis.error.ExceptionList().create_exception()`, передавая ему UUID исключения, другие атрибуты исключения и атрибут `passed_exception` — указатель на объект C++ исключения `current_exception()`.

Метод `create_exception` ищет UUID в зарегистрированных с помощью `sbis.error.ExceptionList().register()` исключениях. Если найдено — создается конкретный класс исключения, иначе — базовый класс `sbis.error.SerializableException`, в котором сохраняется UUID исключения.

Передача сериализуемого исключения из Python'a

Исключение, переданное в Python при переходе "моста" в C++ транслируется методом `sbis::Python::RethrowException` в C++-исключение.

Возможные сценарии передачи исключения:

1. Если Python-исключение наследуется от `sbis.Error`, то сначала вызываются трансляторы исключений, зарегистрированные с помощью `sbis::RegThrowExceptionCallback`. Если транслятор опознаёт тип Python-исключения, он сам выбрасывает соответствующее C++-исключение, заполняя его данными транслируемого исключения. Иначе, если Python-исключение наследуется от `sbis.SerializableException`, извлекается идентификатор его класса:
 - Если в `sbis::rpc::ExceptionList` зарегистрирован генератор исключений такого класса, ему передаются данные транслируемого исключения, и он выбрасывает соответствующее C++-исключение.
 - Если класс исключений не зарегистрирован, формируется и выбрасывается `sbis::PythonSerializableException`.
2. Если Python-исключение наследуется от `sbis.Warning` - формируется и выбрасывается `sbis::PythonWarning`.
3. Если Python-исключение наследуется от `sbis.KeyboardInterrupt` - формируется и выбрасывается `sbis::PythonKeyboardInterrupt`. Иначе формируется и выбрасывается `sbis::PythonException`.

Остальные исключения транслируются в `sbis::PythonException` с текстом Python-исключения и пустым кодом ошибки.